DineFlow · Technical part

Sprints 1, 2 and 3 explained

What each sprint built, where it lives in the code, and how we checked that it's complete and working.

Sprint 1 · Tables & QR | Sprint 2 · E-menu | Sprint 3 · Live orders

Prepared by Ayman Alhawari

2026-10-25

Summary

✓ **All three sprints are complete and in the code on `main`.**
Each sprint has its own commit, all its key files are in the latest version, and every backend test passes.

3 / 3	241 / 241	81	0
Sprints complete	Tests passing today	Tests added in Sprints 1–3	Failing tests

What is DineFlow?

A guest scans the QR code on the table, opens the menu on their phone, orders, pays and rates the visit. The restaurant sees orders, payments and reviews live in its dashboard.

Stack

Part	Technology
Backend	ASP.NET Core 8 Web API, Clean Architecture, JWT, SignalR
Database	SQL Server 2022 through EF Core 8
Frontend	Angular 21 + Tailwind CSS 4, with two apps: `guest` (the guest's phone app, English and Arabic with RTL) and `merchant` (the restaurant dashboard)
Tests	xUnit; services are tested against an in-memory SQLite database

Backend layers

`Domain`	`Application`	`Infrastructure`	`Api`
Entities and business rules (e.g. Order, Menu, QrToken)	Services: use cases such as placing an order or scanning a QR	Database, migrations, cryptography, image storage	Controllers, authorization policies, SignalR hubs

Dependencies always point inwards: the Api calls the Application, and the Application depends on the Domain. The Domain knows nothing about the database.

Sprint 1 · `commit 910ea5e`

Tenants, venues, tables, QR codes and guest sessions

Epics: E1 (tenant and venue foundation) · E2 (QR and table sessions)

Goal: build the foundation. A restaurant has venues and tables, each table has a QR code, and scanning it starts a guest session with no account needed.

What was built

Part	What it does	File
Entities	`Tenant` (the company), `Venue` (a branch), `DiningArea` (a hall), `ServiceLocation` (table, counter, hotel room, pool…), `StaffUser`	`Domain/Venues/`
QR token	The token itself is never stored. Only its SHA-256 hash is saved, so a leaked database can't be turned into working QR codes	`QrToken.cs`
Token generation	Generated with HMAC-SHA256, 27 characters (160 bits). Can't be guessed and isn't sequential	`QrTokenService.cs`
Guest session	`GET /v1/public/qr/{token}` checks the token, creates a `GuestSession` and gives the guest a short-lived JWT	`GuestService.cs`
No token leaks	Unknown, disabled and retired tokens all return the same `qr_invalid` error, so an attacker can't tell which tokens exist	`GuestService.cs`
Tenant isolation	Every entity implements `ITenantOwned`, and every query is filtered by `TenantId` automatically, so no restaurant can see another's data	`AppDbContext.cs`
Merchant sign-in	JWT with `Merchant` and `MerchantManager` policies, and rate limits on sign-in and QR scans	`AuthService.cs`
Table management	Add areas and tables (one at a time or in bulk), and rotate, disable or enable a table's QR code	`TableService.cs`
Errors	Every error comes back as ProblemDetails with a stable `code`, e.g. `venue_unavailable` or `invalid_credentials`	`Api/Common/`

How a QR scan works, step by step

1. The guest scans the code, which opens the link and sends the token to `/v1/public/qr/{token}`.
2. The server checks the token is well formed, hashes it and looks the hash up in the `QrTokens` table.
3. It checks the token is active, the table and venue are active, and the company isn't suspended.
4. It creates a `GuestSession` linked to the table and picks the language (English or Arabic).
5. It returns a JWT with the venue's details (name, logo, colour, currency) and the table's.

Guest app

The landing page after scanning, in English and Arabic with RTL, plus "invalid QR" and "venue unavailable" screens. The session is saved, so refreshing the page doesn't need a new scan.

Merchant dashboard

Sign-in, the **Tables & QR** page (add areas and tables; print, download and rotate codes) and **Settings** with a live preview of the guest screen.

Tests: 28, including tests that one restaurant can't see another's data.

Sprint 2 · `commit 3966f0e`

Menu management and the guest e-menu with modifiers

Epics: E4 (menu foundation) · E31 (e-menu enhancements)

Goal: the restaurant builds its menu in the dashboard, and the guest browses it, picks modifiers and adds items to the cart.

What was built

Part	What it does	File
`Menu`	A venue can have several menus (e.g. "All day" and "Breakfast"). Each has days and opening hours, which can run past midnight, and is published or a draft	`Menu.cs`
`MenuCategory`	Menu categories, with one level of subcategories	`MenuCategory.cs`
`MenuItem`	An item: name in English and Arabic, price in minor units (JOD 12.500 is stored as the integer `12500`), photo, sold-out flag, 14 allergens and tags	`MenuItem.cs`
Modifiers	Modifier groups (`ModifierGroup`), each with a minimum and maximum number of picks and default options. E.g. "Choose a size" is required and "Extras" is optional	`ModifierGroup.cs`
Guest menu	`GET /v1/public/sessions/{id}/menu` returns the published menus and whether each can be ordered from right now, in the venue's local time	`GuestMenuService.cs`
Menu admin	Create, edit and delete (CRUD) menus, categories and items, and reorder them. Any staff member can mark an item sold out; everything else is for managers	`MenuAdminService.cs`
Photo upload	JPEG, PNG or WebP up to 5 MB. The type is checked from the file's content, not its extension. Served from `/media` with long caching	`MenuImageService.cs`
Demo data	Bilingual "All day" and "Breakfast" (07:00–11:30) menus, with one sold-out item to try	`DemoMenuSeed.cs`

Example: how menu schedules work

"Breakfast" is set to 07:00–11:30. A guest who opens the menu at 2 pm can browse it but can't order from it. The server works this out in the venue's time zone (Amman), not the time on the guest's phone. Hours such as 22:00–02:00 mean the menu runs past midnight.

Guest app

The menu page has a menu switcher, search, category chips, grid or list view and sold-out states. The item sheet shows required and optional modifiers with a live price, then a cart and a review page.

Merchant dashboard

The **Menu** page has editors for menus, categories, items and modifier groups, photo upload, a sold-out toggle and reordering.

Tests: 32 new (60 in total at the time).

Sprint 3 · `commit a5bfed9`

Server-priced orders, live order queue and staff alerts

Epics: E6 (cart and order) · E10 (staff notifications)

Goal: the guest sends an order, the server prices it, and the kitchen and staff see it instantly.

What was built

Part	What it does	File
Pricing	The one place totals are calculated. Service charge applies to the subtotal; tax applies to subtotal + service charge. Supports tax-inclusive prices or tax added on top	`OrderPricing.cs`
Cart check	The server prices the cart and reports a problem per line: item sold out or removed, menu not served now, modifiers changed or outside the allowed picks	`CartQuoteService.cs`
Order	`Order` and `OrderLine` keep a snapshot of names and prices at ordering time, so later menu edits never change an old order. Numbers start at 1001 per venue	`Order.cs`
No duplicates	Each order carries a `clientRequestId` with a unique index. If the connection drops and the guest resends, the same order comes back instead of a second one (idempotency)	`GuestOrderService.cs`
Price changed	If the total the guest saw differs from the server's, it returns `409 price_changed` with the new quote. Nothing is ordered at a price the guest didn't accept	`GuestOrderService.cs`
Order status	Staff move the order along, and any transition that isn't allowed is refused (see the life cycle below)	`MerchantOrderService.cs`
Live updates	SignalR: staff connect to `/hubs/merchant` and get `orderPlaced` and `orderChanged`; the guest connects to `/hubs/guest` and tracks their order live	`Hubs/OrderHubs.cs`

Order life cycle

`Submitted` → `Accepted` → `Preparing` → `Ready` → `Completed (Served)`

`Submitted` → `Rejected + reason`

An order can only be rejected while it's new. After it's accepted, staff can skip steps. The value `Completed` shows on screen as "Served".

Pricing example (tax-inclusive prices, as in Jordan)

Items subtotal	JOD 20.000
10% service charge	+ JOD 2.000
Total the guest pays	JOD 22.000
Of which 16% sales tax (shown, not added)	JOD 3.034 = 22.000 − (22.000 ÷ 1.16)

Guest app

The order review page (server prices, charges, a note for the kitchen, a "Place order" button that's safe to retry) and an orders page with a live tracker and any rejection reason.

Merchant dashboard

A live **Orders** board with New, Accepted, Preparing and Ready columns, a chime, a toast, a nav badge and a tab-title count, and a late-order flag. **Settings** gains service charge and VAT.

Tests: 21 new (81 in total at the time).

Ground rules the code follows

1. **Money is never stored as a float.** We use `long` minor units plus a currency code. The Jordanian dinar has 3 decimals, so one dinar is stored as `1000`.
2. **The server prices the order**, not the guest's phone. The guest only confirms the price the server worked out.
3. **Every write carries an idempotency key** (`clientRequestId`), so resending is safe.
4. **QR tokens are never stored**, only their hash, and they can be rotated at any time.
5. **Each restaurant's data is isolated automatically** by the tenant filter on every query.
6. **An order keeps a snapshot of prices and names** from ordering time, so menu changes never alter old orders.

How we checked the sprints are complete

Check	Result
Git history	Each sprint has a commit on `main` : `910ea5e` , `3966f0e` and `a5bfed9`
Files present	All the key files for the three sprints are in the latest version, e.g. QrToken, GuestService, Menu, ModifierGroup, Order, OrderPricing, CartQuoteService, OrderHubs
`dotnet test`	241 of 241 tests pass, none fail
Not checked	The Angular tests (`npm test`) and a manual run in the browser

Running a demo for the team

1. Start the API. It creates the database and seeds a demo venue, "Demo Bistro" in Abdoun, Amman:

```
dotnet run --project DineFlow/DineFlow.Api --launch-profile http
```

2. Start the guest and merchant apps:

```
npm --prefix web ci
npm --prefix web start
```

3. Demo steps:

1. Open the merchant dashboard at http://localhost:4300 and sign in. The sign-in details are in `appsettings.Development.json` under `Seed:Owner` .
2. **Sprint 1:** go to **Tables**, show the tables and QR codes, then click **Open** on any table to open the guest screen.
3. **Sprint 2:** click **View Menu** and show the categories, search and modifier sheet. Then mark an item sold out in the dashboard and reload the guest menu.
4. **Sprint 3:** add items and place the order. It appears instantly under **Orders** with a chime. Move it through the statuses and watch the guest's tracker update live.